Lecture 21: Classification

CMSC/DATA 320: Introduction to Data Science

Lecturer: Anna Evtushenko (annae@umd.edu)

Announcements

- PA5, WA5 were due Sunday, April 12
- PA6, WA6 out Tuesday, April 14

Classification

Reminder: Classification

Classification: A supervised machine learning task where the algorithm builds a model to predict what class an input belongs to.

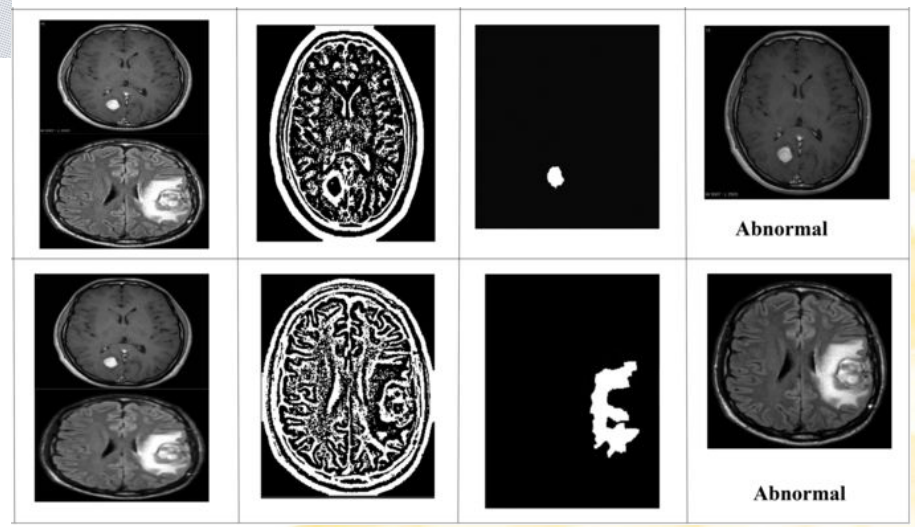
Goal: Assign categories/classes/labels to data points based on patterns discovered in the training data.

Output: Categorical outputs. The dependent variable is assigned a label/class/category.

Example: Categorizing loan applicants (Paid Back Loan) into "Yes" or "No" labels.

Real Life Classification Examples

- SPAM Detection
- Handwriting Recognition
- Fraud Detection
- Cancer Identification
- Credit Risk Assessment
- Product Categorization
- … many many more!



Classification vs Regression

Regression: Predict **continuous** **numerical** values.

- Examples:
  - What will be the price of gasoline?
  - What will be the temperature of this room?

Classification: Predict a **discrete** **categorical** values.

- Examples:
  - Will this person pay back their loan (yes/no)?
  - Is this email SPAM (yes/no)?

Lazy Learners Vs. Eager Learners

Eager learners

- Build a model from the training data **before** making predictions on unseen data
- Ex. Decision Trees

Lazy Learners

- *Don't* create a model from the training data first.
  - Simply remember all the training data, then at prediction time search for a nearby neighbor to make a prediction.
- Ex. K-Nearest Neighbor

Classification, so far

We have motivated our understanding of classification thus far by looking at two algorithms:

1. K-Nearest Neighbors (KNN) Classification
2. ID3 Decision Tree Classification

Next, we will look at some other important classification algorithms:

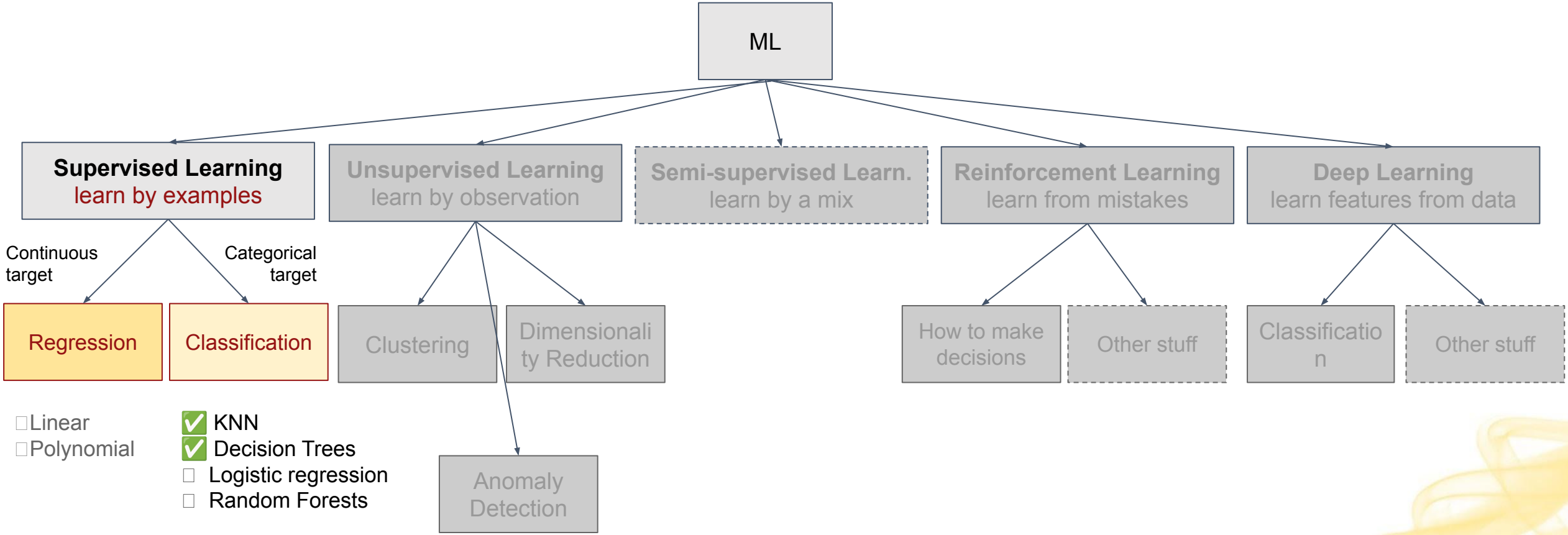
1. Logistic Regression
2. Random Forests

Why do we need more classification algorithms?

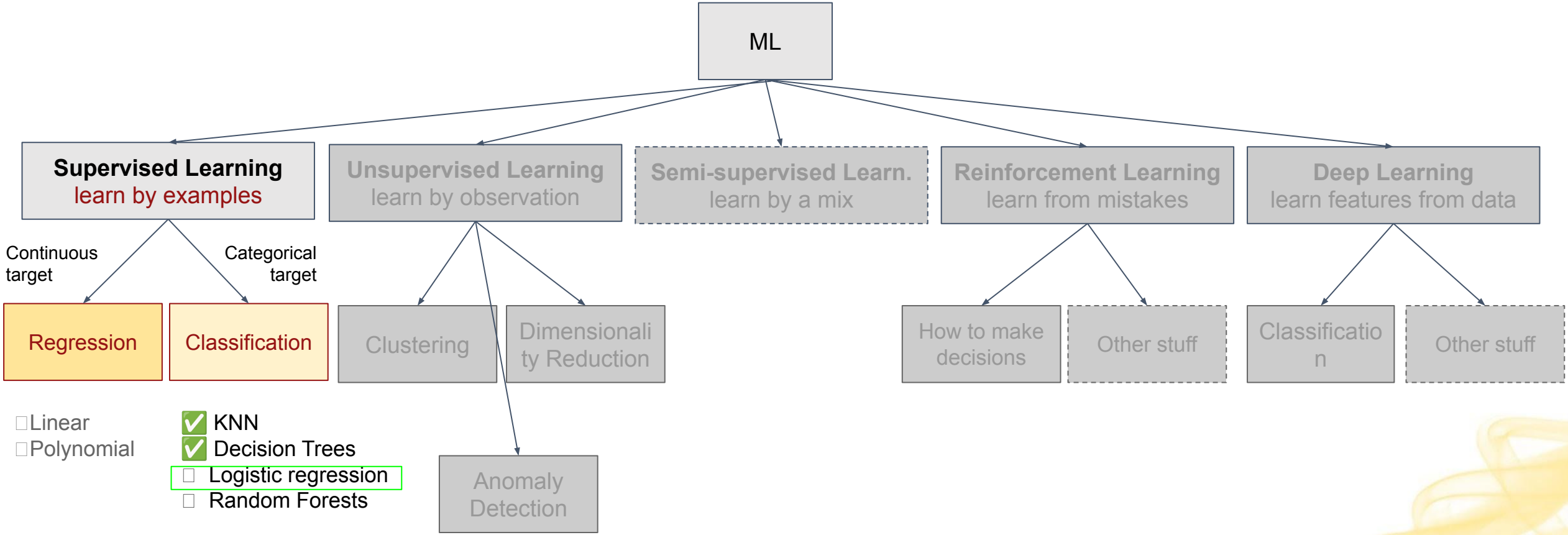
Remember: No Free Lunch Theorem (NFL)

There is no single algorithm that will work well for all tasks.

Main categories of Machine Learning



Main categories of Machine Learning



Logistic Regression

Importance of Logistic Regression

- Logistic Regression is an incredibly powerful ML tool
  - Many, many real-life problems can be boiled down to something Logistic Regression can solve well
  - It is particularly good at Binary Classification, and the most widely used
- Is a baseline supervised machine learning algorithm for classification
  - ex. Logistic Regression is foundational for Neural Networks

Logistic Regression vs Regression

In spite of it having “regression” in the name, Logistic Regression is a ***Classification*** algorithm.

- **What is Regression, in general?**
  - A way to estimate the behavior of an output variable (dependent variable) in response to changes in a different input variable (independent variable) using statistical techniques.
  - In ML, when we say “regression”, we are typically talking about regression for a ***continuous, normal output variable***
- **Regression**: regression for a **continuous, normally distributed output variable**
  - **Linear**: tries to fit the relationship to a line
  - **Polynomial**: tries to fit the relationship to a curve (a polynomial function)
  - Others…
- **Logistic Regression**: regression for a **discrete (categorical) output variable**
  - Typically **Binary Classification**!
    - Ex. true/false, yes/no, spam/not spam, loan/no loan
  - Also there is multinomial and ordinal (*won’t cover*)

Logistic Regression (LR, or Logit Regression)

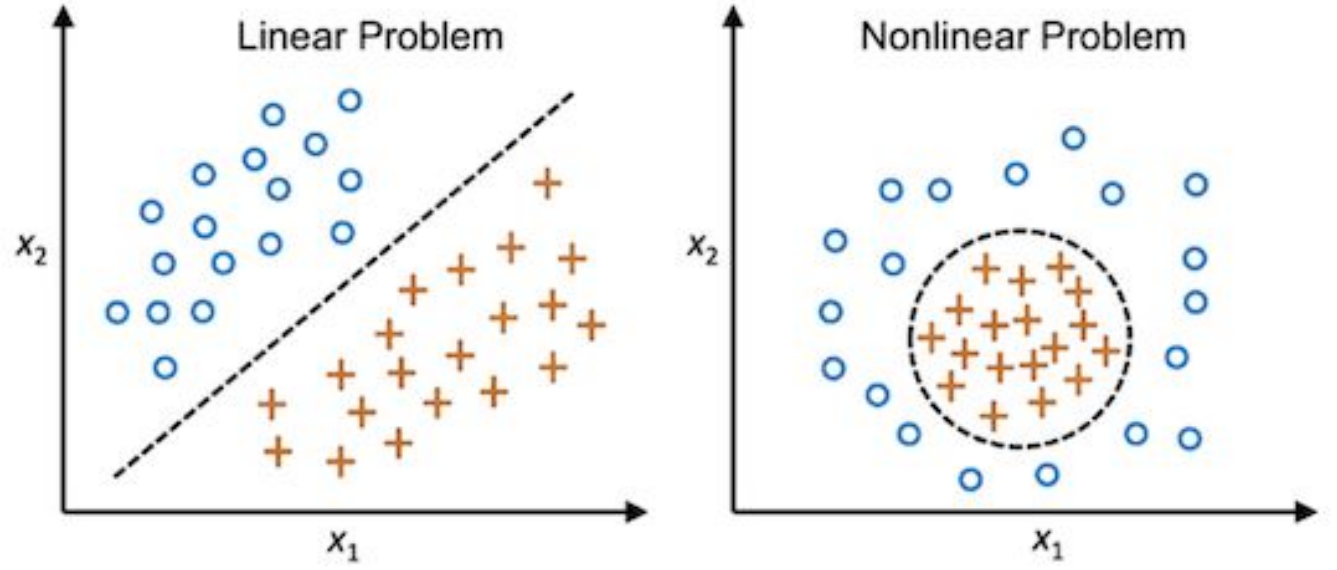
Classification algorithm that predicts a class label by modeling its probability using log-odds as a linear combination of one or more features.

- **Main use case**: binary classification
- **Goal**: find the decision boundary which separates 2 classes
  - For 2 dimensions: straight line $ax + by + c = 0$
  - For 3 dimensions: plane $ax + by + cz + d = 0$
  - For >3 dimensions: hyperplane $w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$
  - For simplicity, we'll just call all of these a "line"
- **Key Assumption**: Linear Separability *(will discuss next)*
  - Actually fairly rare in the real-world
  - But close enough is often good enough
  - LR performs well for linearly separable classes

Linear Vs. Non-Linear Separability

For a binary classification problem:

- If a straight line (or a hyperplane for n-dimensional space) can divide the two classes, we have a **linearly separable** problem.
- If not, we have a **non-linearly separable** problem.
- ex. In 2D:



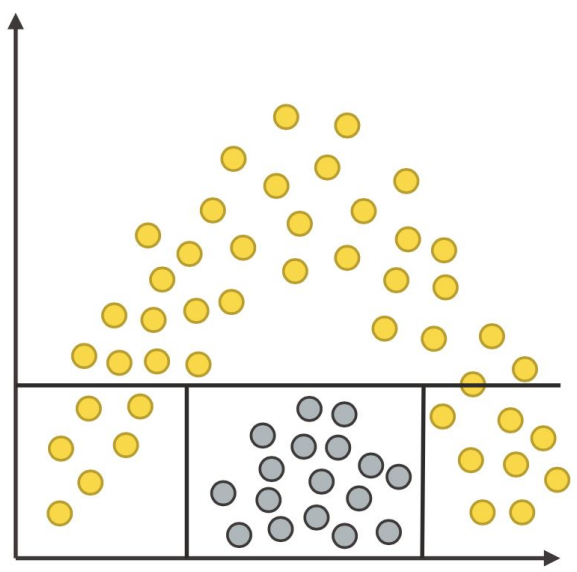
Can’t we just use Decision Trees?

Remember: No Free Lunch Theorem (NFL)

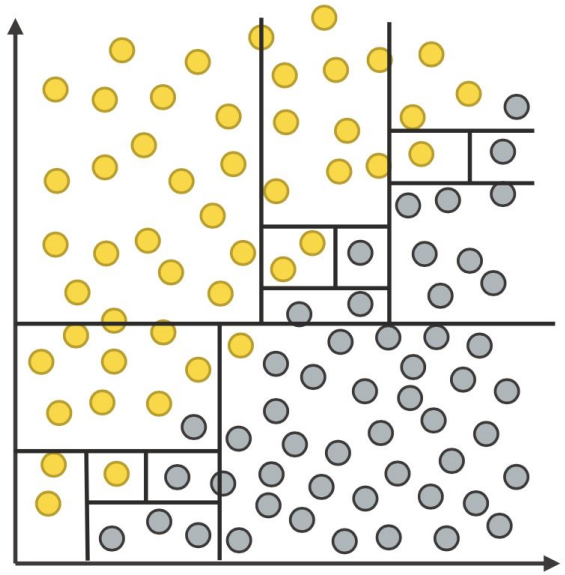
There is no single algorithm that will work well for all tasks.

Comparison of Decision Boundaries - DT vs. LR

Decision Trees

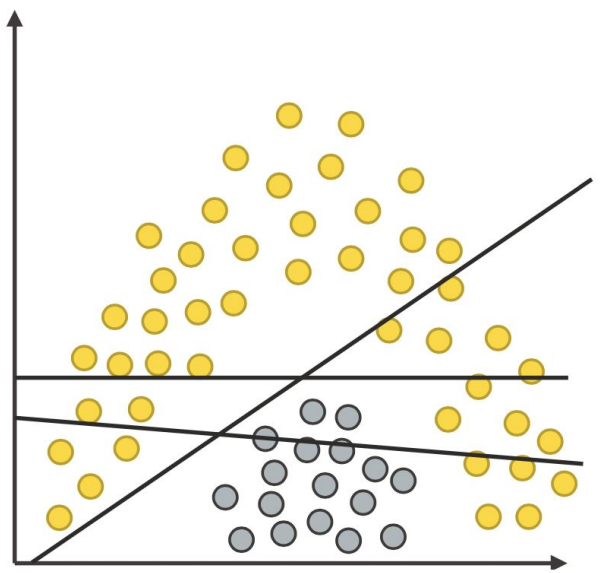


👍

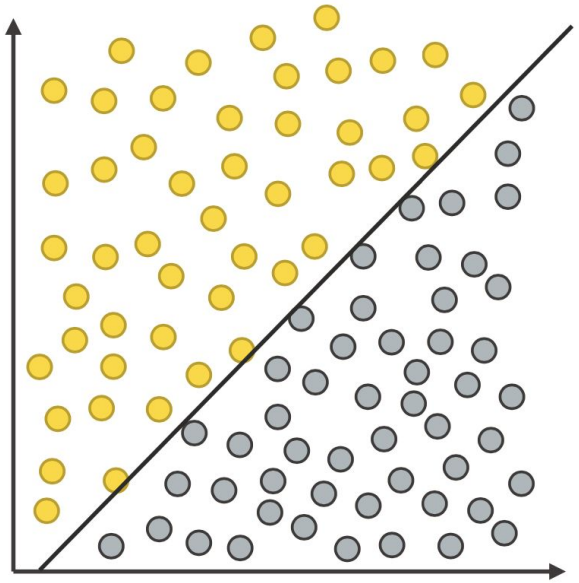


👎

Logistic Regression



👎



👍

We need to find the right tool for the task at hand.

Idea: Assuming Linear Separability, Find the Line

- **Given** a bunch of features (coordinates) $x_1 \dots x_n$
- **Find the boundary** given by equation $w_o + w_1 x_1 + w_2 x_2 + \dots + w_n x_n$
- **Where**:
  - $w_1 \dots w_n$ are the weights that tell us how much each feature contributes to the separation boundary
  - w_o is a constant weight that is the intercept (like b in y = mx + b)
  - We'll let x_o be a dummy "feature" that always equals 1 so that we can rewrite this as:

$$\sum_{i=0}^{n} w_i \cdot x_i = 0$$

- **Find the optimal values** of $w_o \dots w_n$ that will give us the "best" line to separate the classes in the training data

How do we find the weights? 🏋️

- We can rewrite the equation as the multiplication of the feature vectors with a weight vector:

$$W^T \cdot X = \sum_{i=0}^{n} w_i \cdot x_i$$

- Now we have an equation that we can learn to optimize.
- We can utilize an **optimization algorithm** to do this
  - Common one we'll use: **Gradient Descent**
  - It is customary to use θ to represent estimates for the weights instead of W:

$$\theta^T x$$

Gradient Descent

Gradient descent is an iterative optimization algorithm used to **find the minimum of a function**.

- Finds the minimum point of a function by iteratively following the direction of steepest descent
- Used to minimize a cost function
  - Starts with some initial values for the weights, then iterates.
  - Increases or decreases the parameters to find the next cost function value.
  - Repeats until a minima is found.
- We'll go into Gradient Descent in much more detail when we discuss Regression
  - For now, just know that it is **a technique to minimize a cost function**

What is a Cost Function?

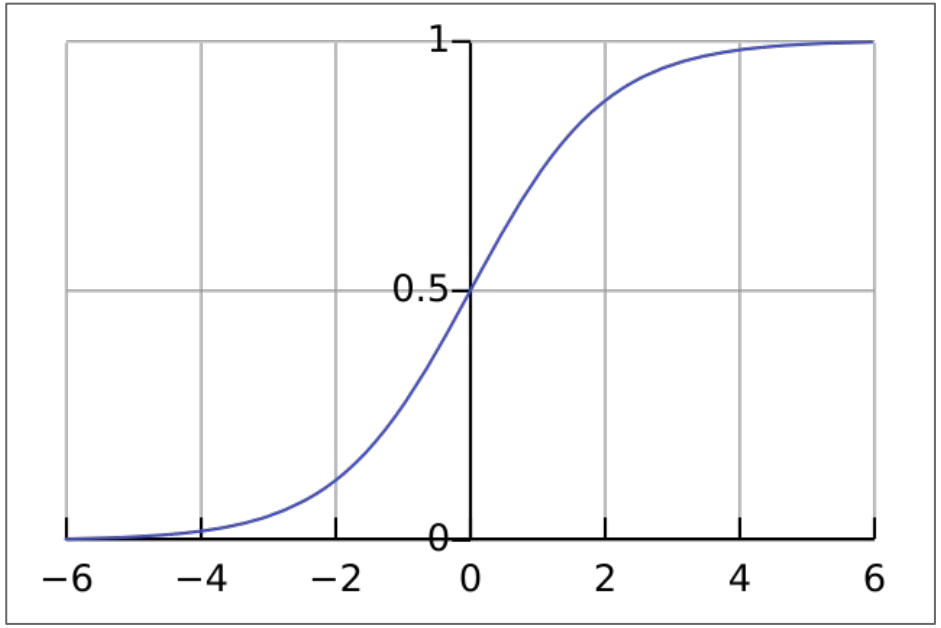
A function to measure how well an ML model is performing.

- Calculates the difference between a model's predictions and the actual values
  - This is what we were doing by hand when we talked about Model Evaluation
- Gives us a way to minimize cost/errors
- Requires an optimization objective
- Examples
  - For **Decision Trees**, **Entropy** was our cost function - we wanted to minimize it by maximizing **Information Gain**
  - When we talk about **Regression**, this will be **Mean Squared Error**
  - For **Logistic Regression**, we use a modeled **cost of the Logistic Function** (Sigmoid Function)

Logistic function (Sigmoid function)

A useful mathematical function with some helpful properties:

- Domain: [-∞,∞]
- Range: (0, 1) - bounded output
- s(0) = 0.5 - symmetric about (0, 0.5)
- Continuous
- Differentiable
- Monotonically increasing
- Can pretty much call anything:
  - with x>10 a 1
  - with x<-10 a 0



$$s(x) = \frac{1}{1+e^{-x}} = \frac{e^{x}}{1+e^{x}}$$

But wait, we said we were looking for a line?!

Decision Boundary vs Logistic Function

- The **Decision Boundary** is a line where the probability of belonging to one class is equal to the probability of belonging to the other class
  - Data points on one side of the line are predicted to be in one class
    - unless we decide something else
- The **Logistic Function** (Sigmoid) gives us a way to fit the decision boundary line to the data
  - Can use to minimize the error between the predicted probability and the actual class label
- Let's look at how.

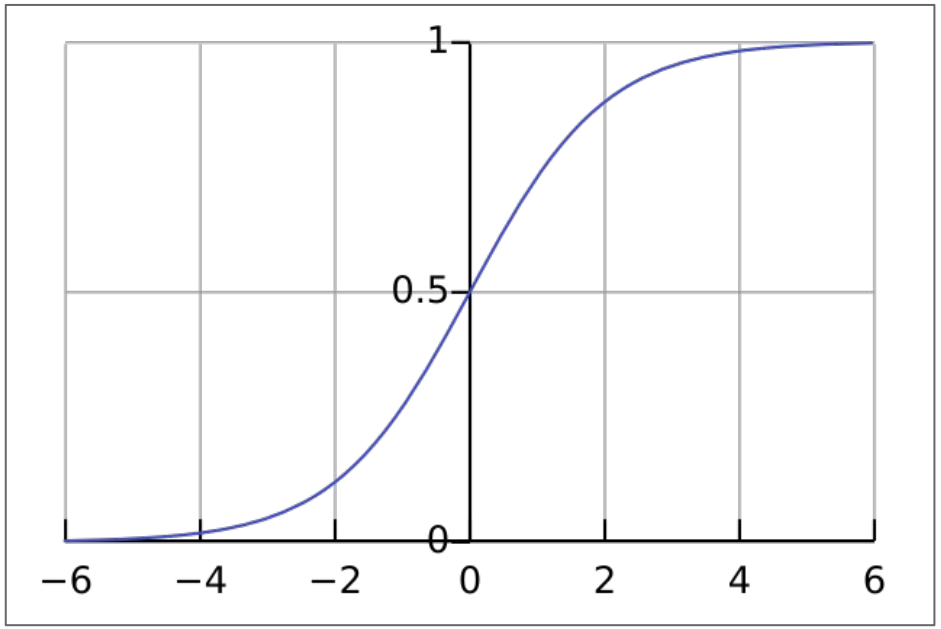
Approach

- Rather than model the **output** (predict the class label), model the ***probability*** **of the output** (the probability that the class label is a particular value for a given observation).
  - Given observation data X, predict output Y
  - Learn the function $P(Y|X) \rightarrow$ a probabilistic classifier
- Probabilities are always between 0 and 1
- The sum of probabilities for all classes is 1
  - Handy!
  - For binary classification, we just need to model the probability of one class

Logistic Function: Good for representing probabilities

Notice that this function represents values on the range [0, 1].

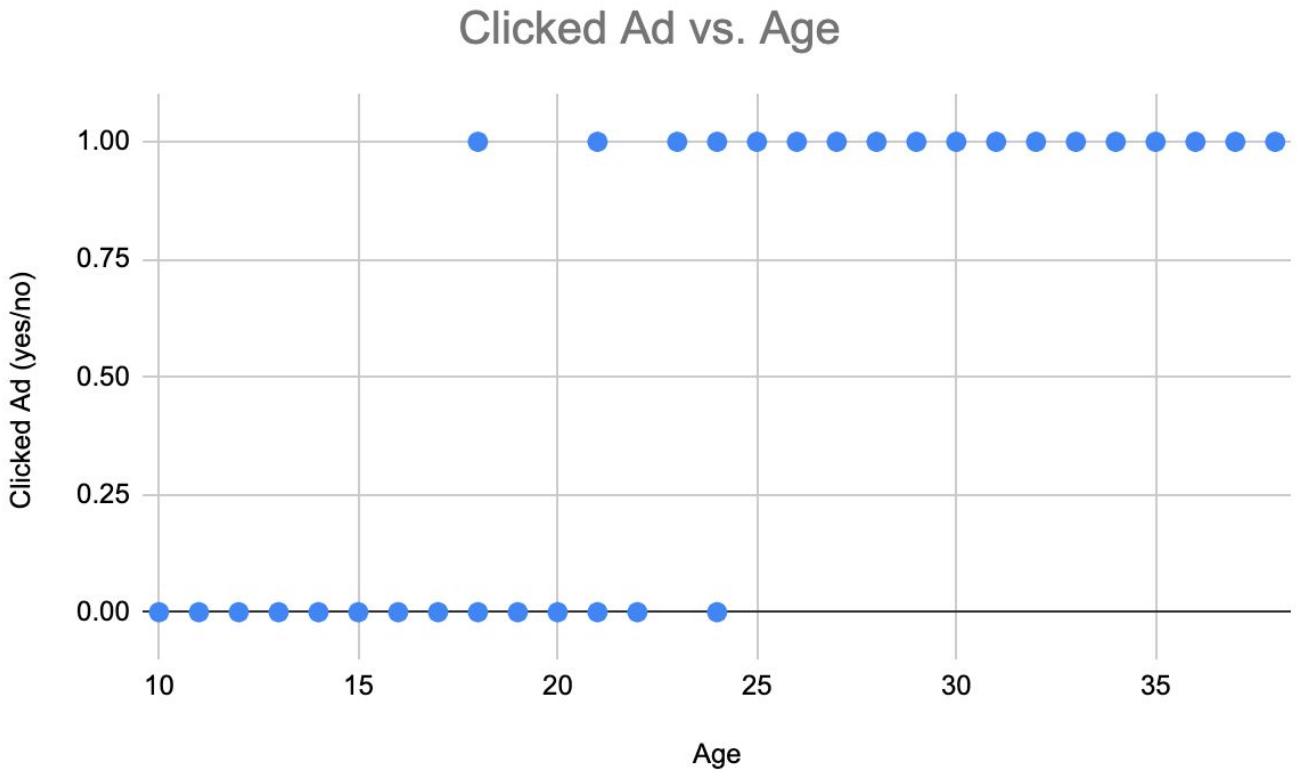
We can use this to **map an input to a probability**.



$$s(x) = \frac{1}{1+e^{-x}} = \frac{e^{x}}{1+e^{x}}$$

Motivating Example

Suppose we have



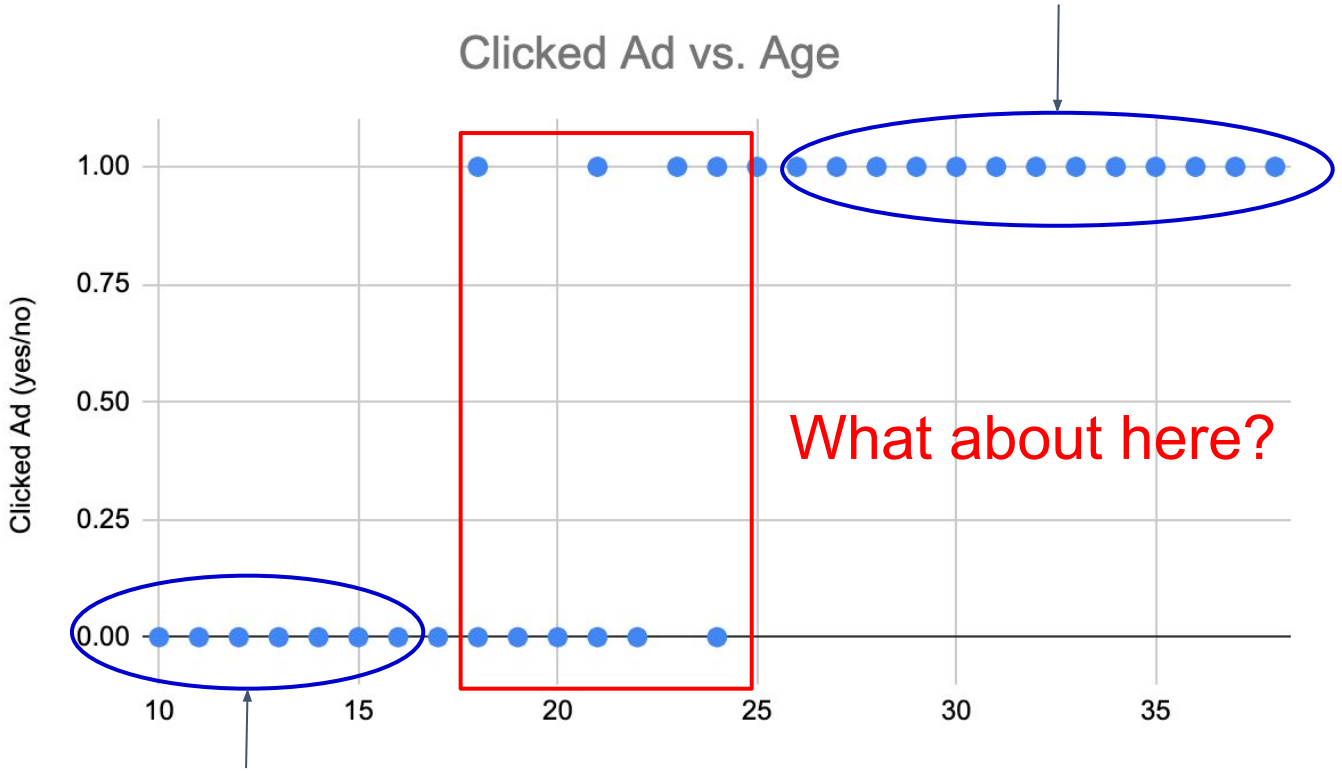
Binary classification problem:

Based on age, will they click on the ad (yes/no)?

Motivating Example

Suppose we have

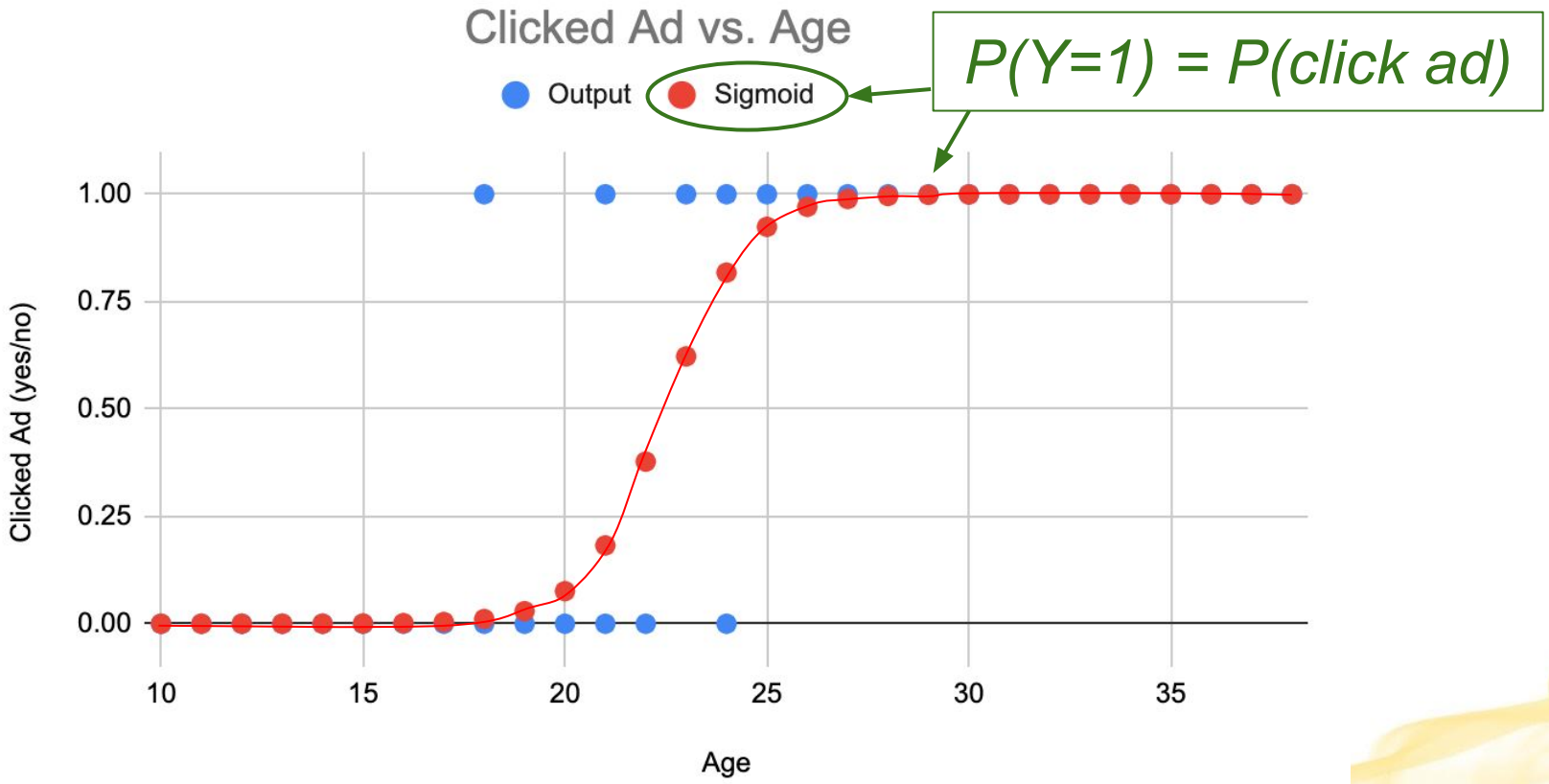
Age > 25, going to click on the ad → P(ad click) = 1



Age < 15, not going to click on the ad → P(ad click) = 0

Motivating Example

We can use the Logistic Function to model the probability that they will click the ad.



The reality of how the logistic function is used

$$z = \sum_{i=0}^{n} w_i x_i = \mathbf{w}^T \mathbf{x}$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$P(y = 1|\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}} \qquad \text{same as} \qquad h_\theta(\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x})$$

$$Cost(h_\theta(\mathbf{x}), y) = \begin{cases} -\log(h_\theta(\mathbf{x})) & \text{if } y = 1 \\ -\log(1 - h_\theta(\mathbf{x})) & \text{if } y = 0 \end{cases}$$

The error

$$Cost(h_\theta(\mathbf{x}), y) = \begin{cases} -\log(h_\theta(\mathbf{x})) & \text{if } y = 1 \\ -\log(1 - h_\theta(\mathbf{x})) & \text{if } y = 0 \end{cases}$$

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(h_\theta(\mathbf{x}^{(i)})) + (1 - y^{(i)}) \log(1 - h_\theta(\mathbf{x}^{(i)})) \right]$$

Now we have something an Optimization Algorithm can use

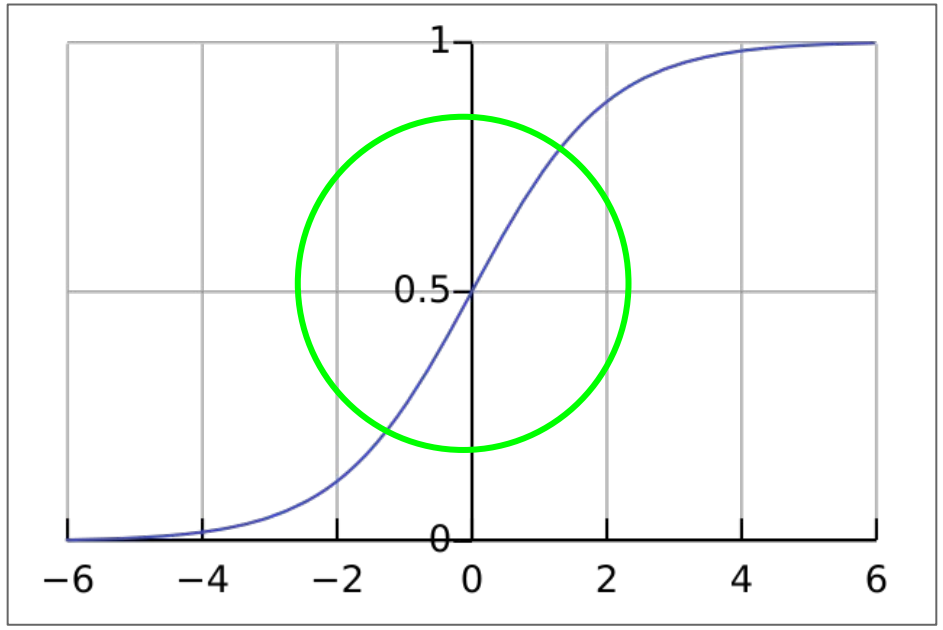
$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(h_\theta(\mathbf{x}^{(i)})) + (1-y^{(i)})\log(1-h_\theta(\mathbf{x}^{(i)}))\right]$$

This will be the function our optimization algorithm will minimize in order to estimate the weights.

Because we are going to into more depth on Gradient Descent in a future lecture, for now we'll just say that it "**optimizes $J(\theta)$**".

What about the middle of the Logistic Function?

How do we choose what class to assign?

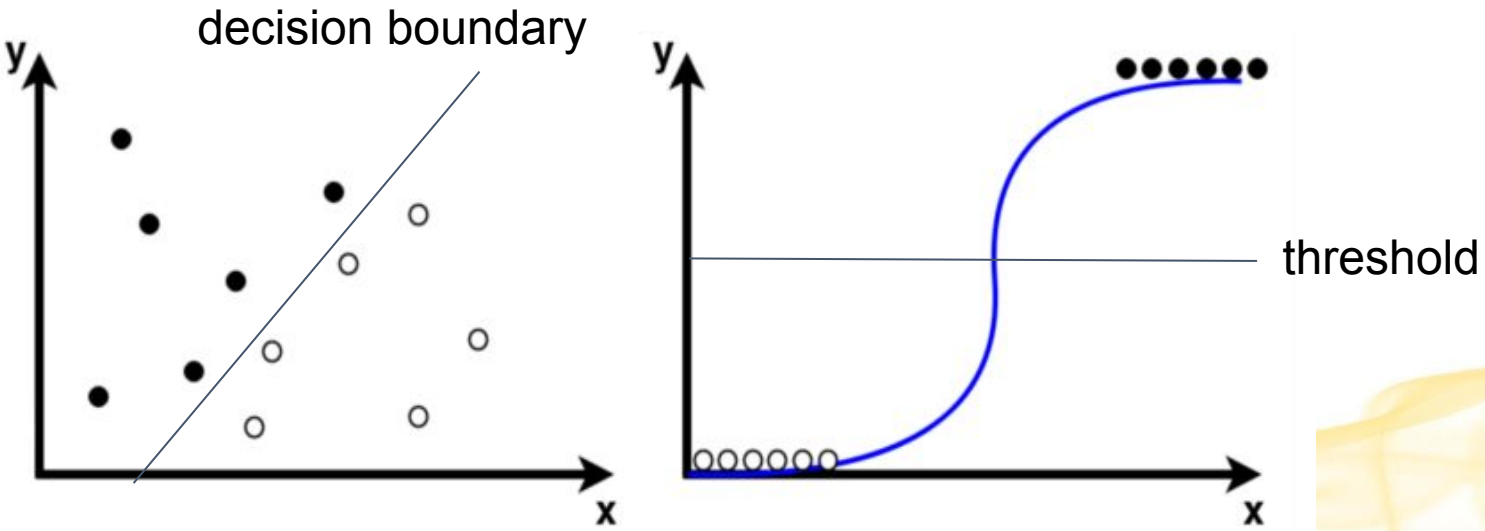


We need to pick a threshold.

Decision Boundary vs. Logistic Function Threshold

- If we map back to the feature domain, then the decision boundary is given by the straight line that is the set of points where $h_\theta(x)$ = *threshold*

$$P(y=1|\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}} \quad \text{same as} \quad h_\theta(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x})$$



Decision Boundary: Threshold Setting

What threshold to choose **depends on the classification problem**.

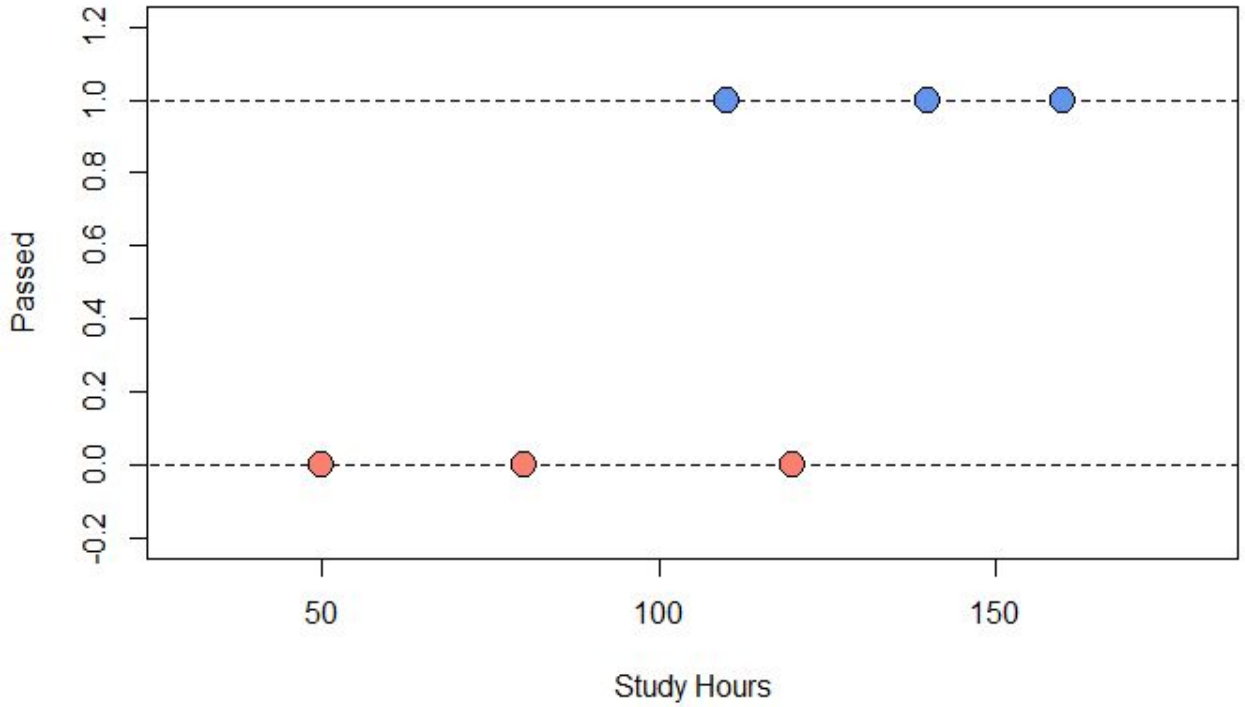
- The obvious choice is (0, 0.5) - middle of the function
  - If P(Y=1) ≥ 0.5, assign the positive class label
  - Otherwise, assign the negative class label
  - This is the default in sklearn
- High Precision
  - If we're averse to False Positives, choose a threshold that gives high precision / low recall
- High Recall
  - If we're averse to False Negatives, choose a threshold that gives high recall / low precision
- If we want High Precision or High Recall, we can try out a bunch of thresholds, calculate those metrics, and pick the one that gives us what we want.

Simple Logistic Regression Example

Suppose we want to predict whether a student will pass an exam based on hours of studying.

Hours Studied	Passed Test
50	0
80	0
110	1
120	0
140	1
160	1

Training Data

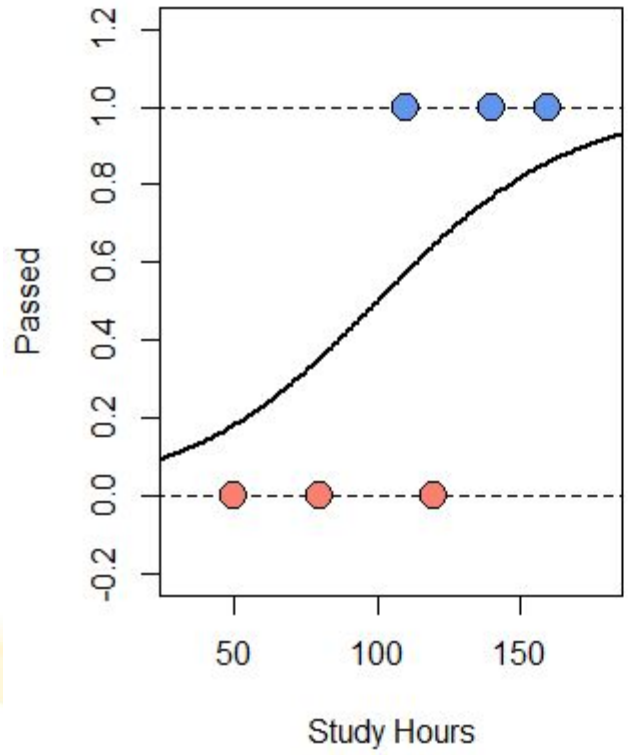


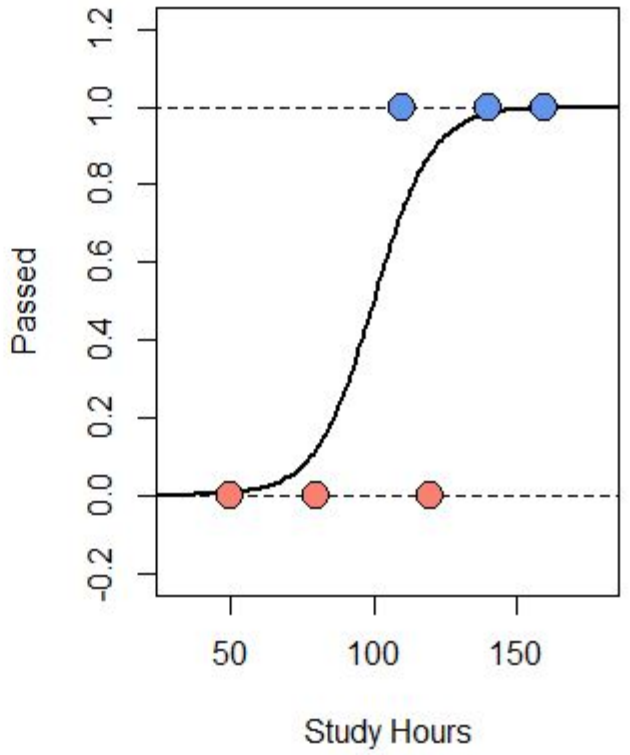
Example credit: https://rstudio-pubs-static.s3.amazonaws.com/534347_d1d5858091154928a404b40f67de2cb1.html

Simple Logistic Regression Example

Suppose now that we have proposed 2 Logistic Regression models:

- Model 1: θx=3−0.03x, $p(X)=h_\theta(x)$
- Model 2: θx=10−0.1x, $p(X)=h_\theta(x)$





Simple Logistic Regression Example

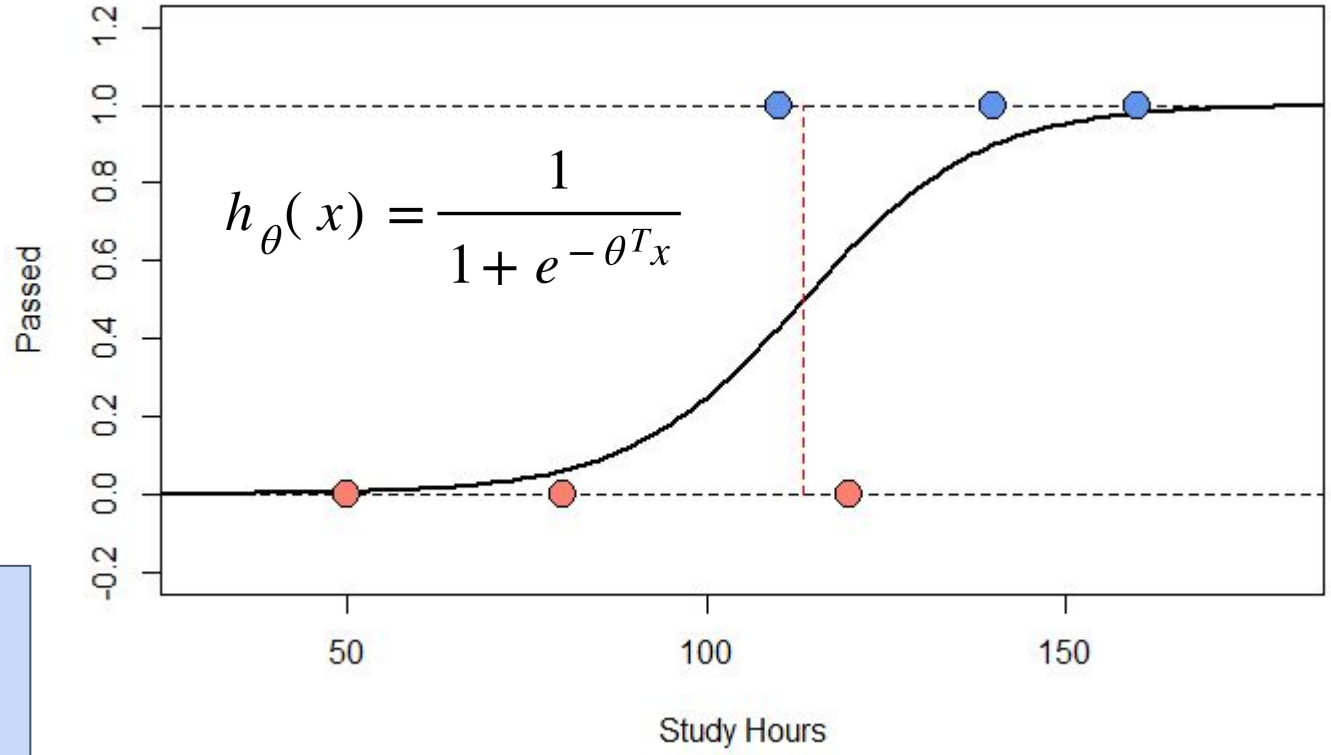
We could calculate their individual likelihoods and compare, but that is cumbersome.

What if we have lots of models? How can we find the best one quickly?

Instead, we use our Optimization Algorithm (ex. Gradient Descent, others) to minimize the error by minimizing our cost function, $J(\theta)$, in order to estimate the best weights.

It pops out $\theta^T x = -9.3 + 0.082 \cdot x$

We can now use this chosen model to make future predictions on.

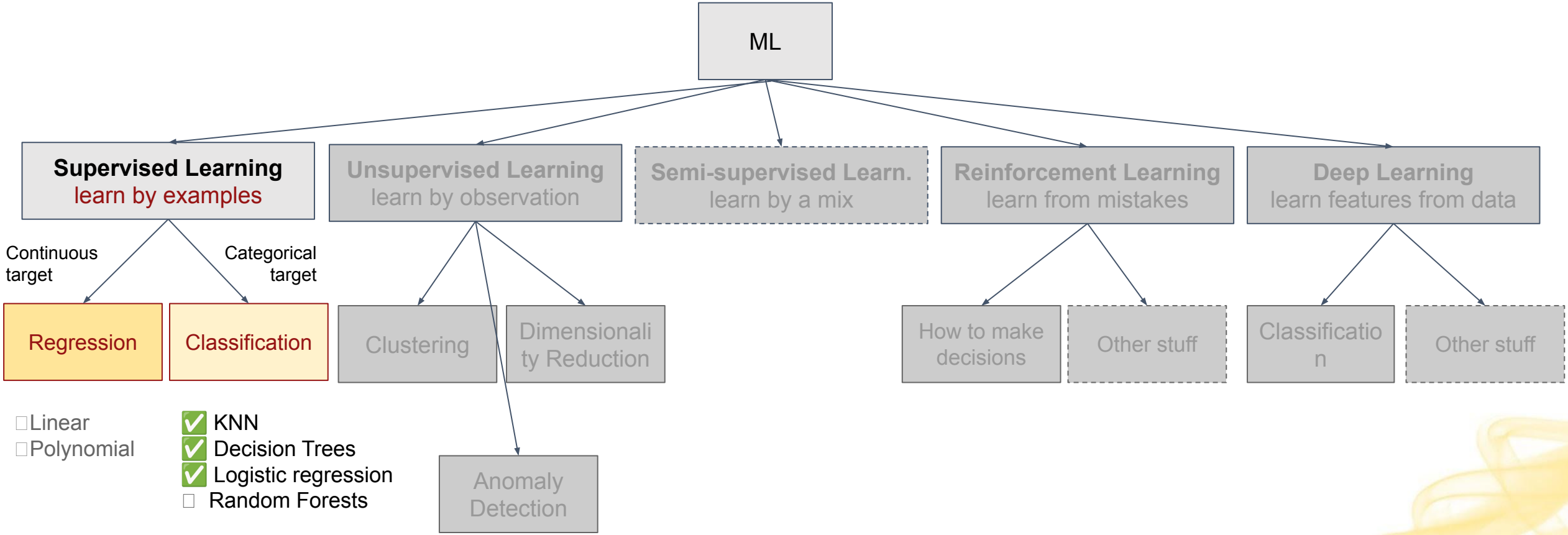


Logistic Regression Python Example

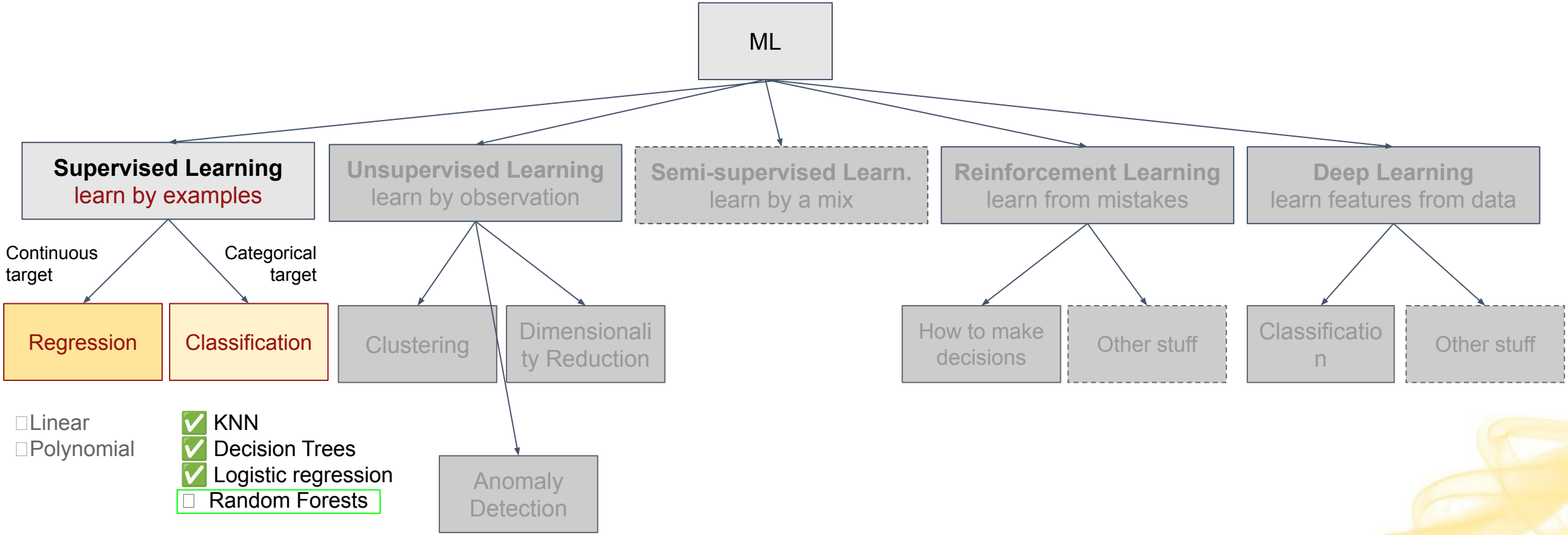
```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Bring in the data
df = pd.read_csv('Iris.csv')
X = df.drop('Species', axis=1)
y = df['Species']

# Split the data into test and train datasets - 80% training and 20% test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=1)

# LogisticRegression
model = LogisticRegression(max_iter=1000, random_state=23)
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Check
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9666666666666667
```

Main categories of Machine Learning



Main categories of Machine Learning



Random Forests

Next: Random Forest Classifier

But before that, let’s go back to “Overfitting” and “Underfitting” a bit.

Recap: We split dataset into training and testing before before training and evaluating a machine learning model. But why do we test?

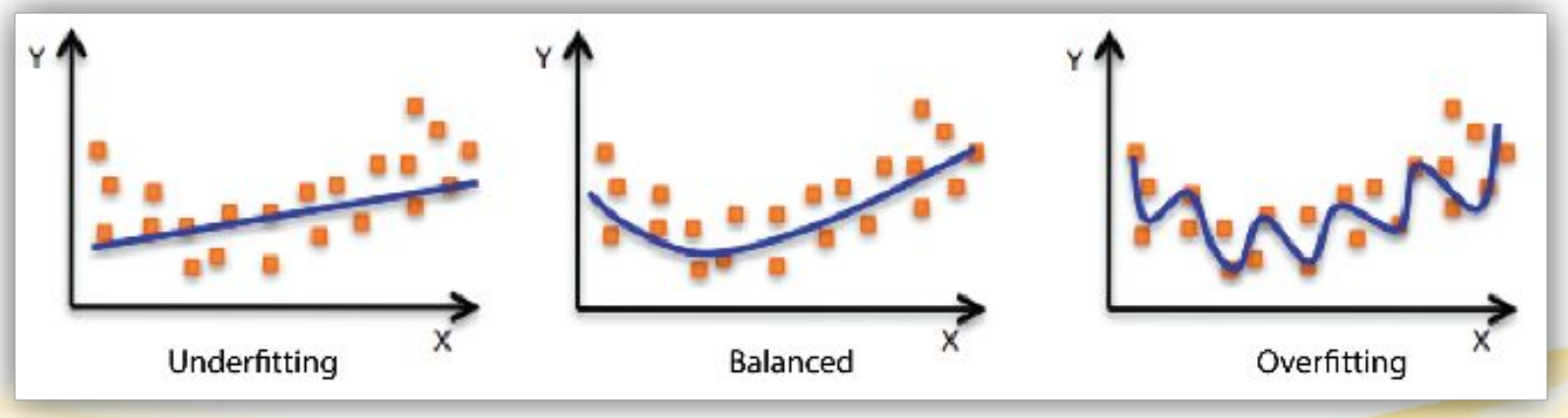
We don't want to use a model that doesn't work! There are many reasons a model might break, but recall that the main ones are:

- **Overfitting**
- **Underfitting**

Recall: Overfitting and Underfitting

Overfitting happens when the model becomes too tuned on the training set, and becomes unable to generalize.

Underfitting happens when the model is too simple/general, and misses patterns in the training data.



Overfitting and Underfitting

Examples of things we can ask:

- Are Decision Trees more prone to overfitting or underfitting?
- What about KNN where K=1? Or K=N?

Overfitting in KNN

In KNN for k=1 (or small value of k): Prone to overfitting.

k=1: Overfitting (too specific): With a small k, the model becomes highly sensitive to noise and outliers in the training data.

- It **reacts too strongly to individual points**, including noise.
- It **memorizes training data points**, capturing noise instead of learning general patterns.
- This results in **poor performance on new, unseen data** as it fails to filter out irrelevant information.

Underfitting in KNN

k=N: Underfitting (too general)!

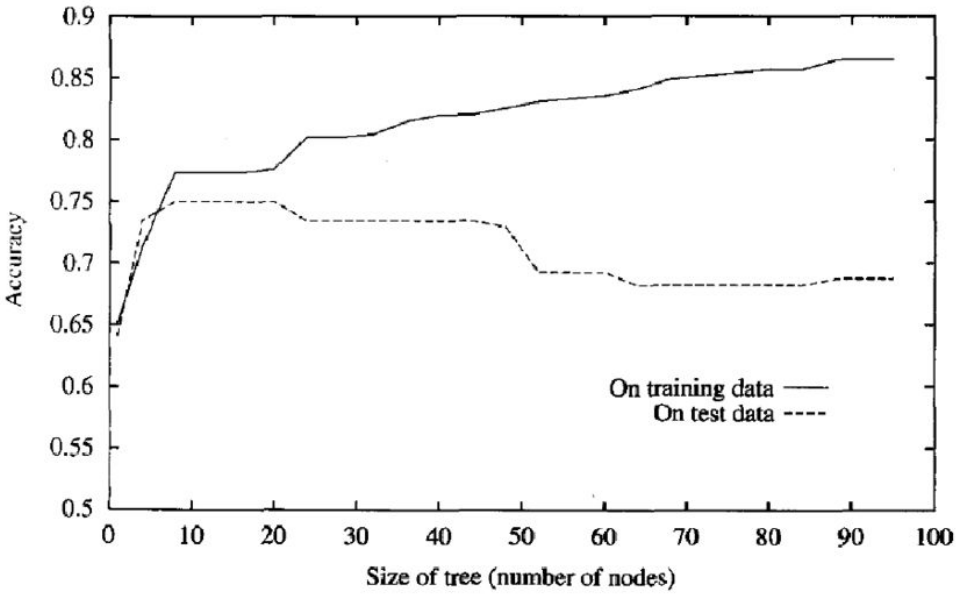
Why? Considers all data points as neighbors, **overly generalized predictions** that will likely underfit the data by not effectively capturing local patterns.

- With k=N, the model uses all the training points for classification. This often leads to underfitting because it averages all points, missing local patterns.
- Results in a simple decision boundary that might not capture the underlying patterns in the data effectively.
- Simplifies the model too much.
- Usually performs poorly because it's too simple to understand the data.

Overfitting in Decision Trees

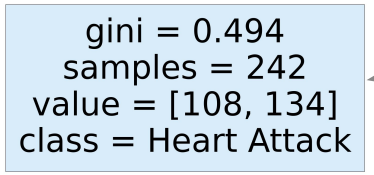
We discussed that Decision Trees are prone to overfitting.

- Without pruning, a Decision Tree will use every observation you give it and effectively memorize the entire training set.
- This creates over-complex trees that do not generalize well.



Underfitting in Decision Trees

We also discussed that if we prune too much, our Decision Trees become too simple and miss key patterns in the data – thus they underfit.



Recall our overpruned tree that was just 1 node that classified everything as a heart attack.

It was an exaggerated example of underfitting.

Dealing with this in KNN

Tune the Hyperparameter k.

For many types of ML tasks, the solution is to tune one or more hyperparameters, and choose what works best.

Dealing with this in Decision Trees

Apply pre-pruning and/or post-pruning techniques.

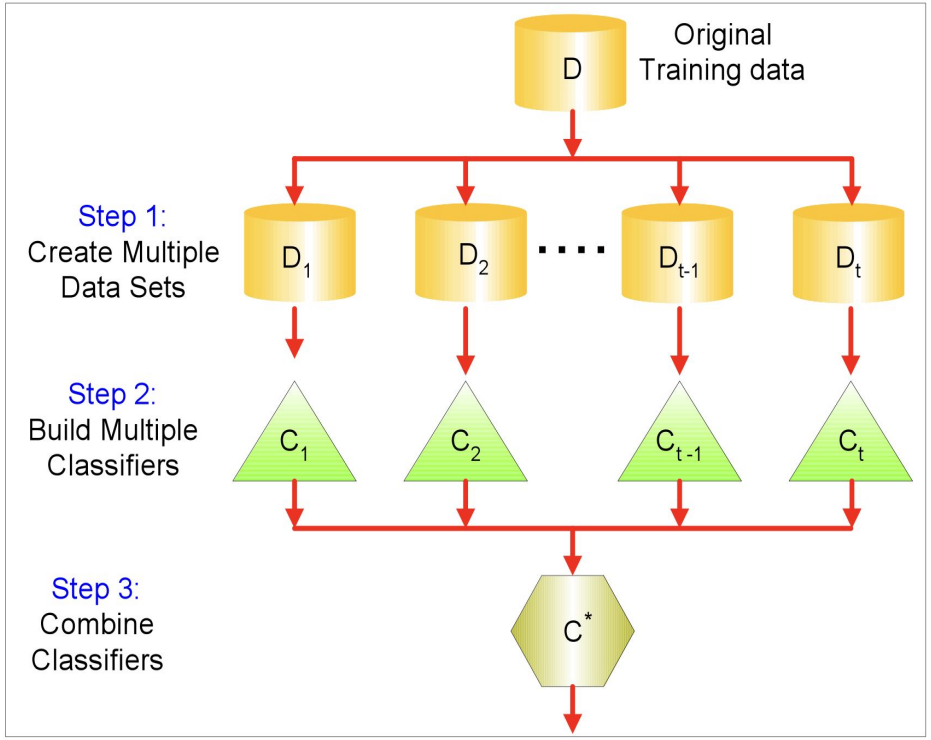
- ex. apply **a depth limit** (fixed depth/ early stopping)
- ex. apply **cost-complexity post-pruning**
- We can try different ones and see how it affects our accuracy on the test set

Or, use ensembles (combinations) of different trees (random forests)

Ensemble methods

Ensemble methods, such as **bagging** and **boosting**, can also mitigate overfitting.

- These techniques combine multiple models to make predictions, reducing the impact of any individual model's biases and errors.
- By leveraging the diversity of these models, ensemble methods enhance the generalization ability and minimize overfitting.

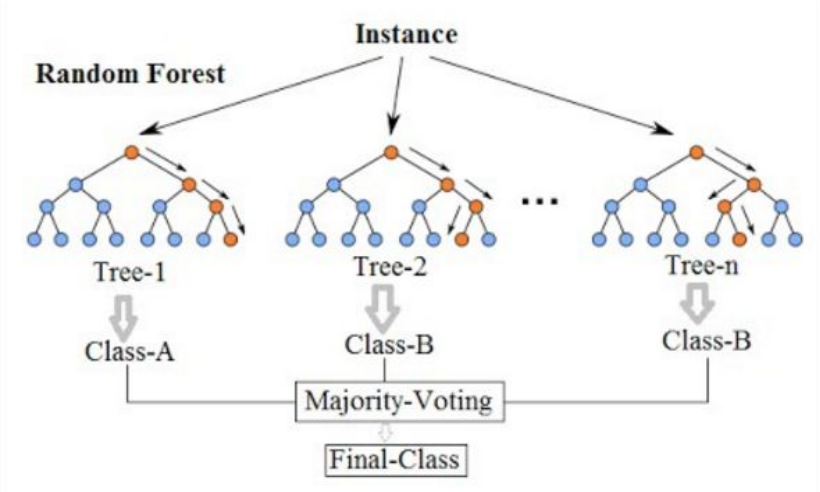


Random Forest

Random Forest is an ensemble learning method primarily used for classification and regression tasks.

- Builds a multitude of decision trees during training
- Merges their predictions to improve accuracy and control overfitting.

Let’s look at how it works.



Key Steps in Random Forest

Bootstrap Sampling (Bagging):

- Create N subsets of the original training data by randomly sampling with replacement.
- Each subset is used to train a different decision tree.

Building Decision Trees:

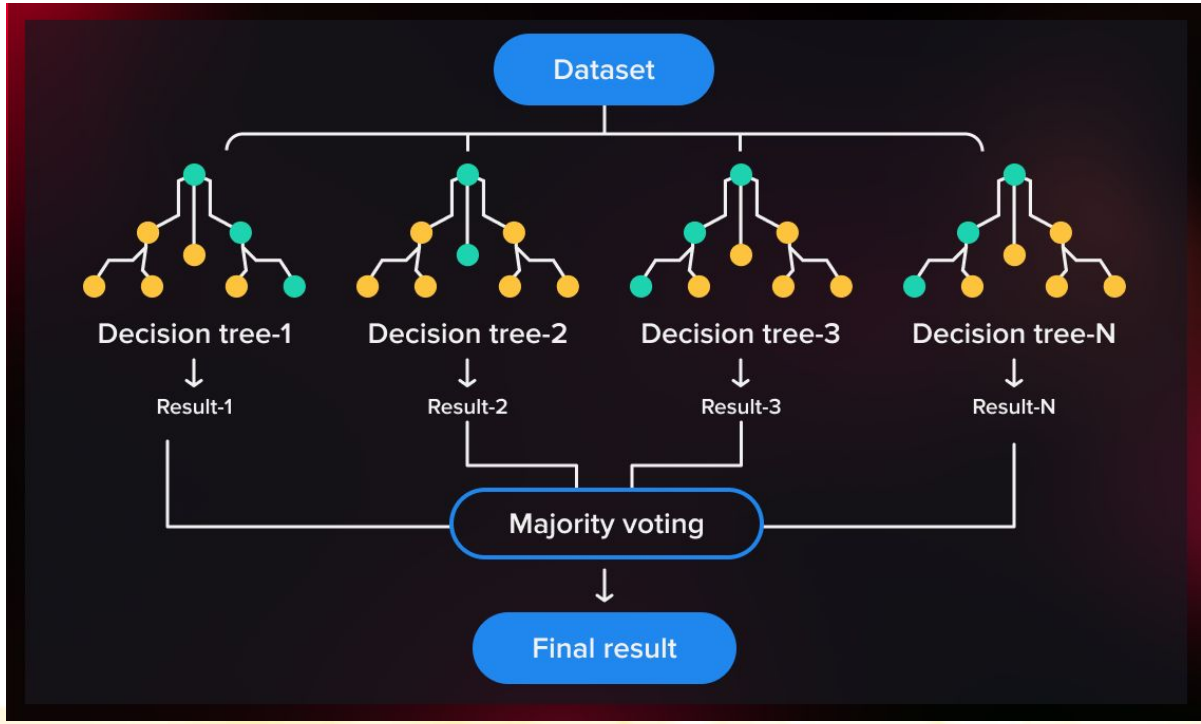
- For each bootstrap sample, a decision tree is constructed.
- Added technique: randomly select features for splitting to increase tree diversity.

- Sampling with replacement

Training Data

Data ID

Original Data	1	2	3	4	5	6	7	8	9	10
Bagging (Round 1)	7	8	10	8	2	5	10	10	5	9
Bagging (Round 2)	1	4	9	1	2	3	2	7	3	2
Bagging (Round 3)	1	8	5	10	5	5	9	6	3	7

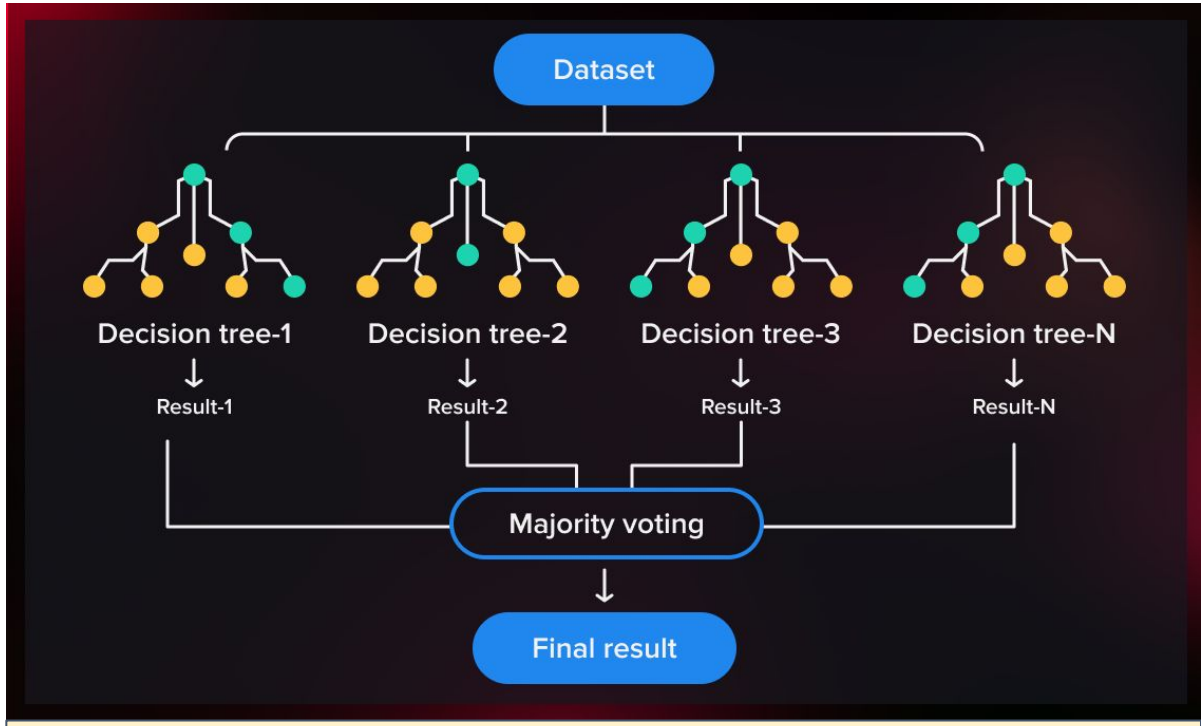


Key Steps in Random Forest

Predictions:

- **Classification**: Use majority voting from all trees.
- **Regression**: Calculate the average prediction from all trees.

❖ If a decision tree is overfit or has locked on to a weird correlation, it gets outvoted



'N' is a hyperparameter that determines the number of trees in the forest.

Usually the higher the number of trees the better the learning. However, adding a lot of trees can slow down the training process considerably, therefore we do a parameter search to find the sweet spot.

Random Forest Python example

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Bring in the data
df = pd.read_csv('heart.csv')
X = df.drop('output', axis=1)
y = df['output']

# Split the data into test and train datasets - 80% training and 20% test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=1)

# Create an Random Forest classifier
model = RandomForestClassifier(n_estimators=50, random_state=42)

# Train the model
model.fit(X_train, y_train)

# Predict Output
y_pred = model.predict(X_test)

# Check
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Example: Comparing a DT to a RF

Single Decision Tree

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Bring in the data
df = pd.read_csv('heart.csv')
X = df.drop('output', axis=1)
y = df['output']

# Split the data into test and train datasets - 80% training and 20% test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=1)

# Create a Decision Tree Classifier
model = DecisionTreeClassifier(criterion="entropy", max_depth=6)

# Train the model
model = model.fit(X_train,y_train)

# Predict Output
y_pred = model.predict(X_test)

# Check
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy:  0.705
```

Random Forest on same data

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Bring in the data
df = pd.read_csv('heart.csv')
X = df.drop('output', axis=1)
y = df['output']

# Split the data into test and train datasets - 80% training and 20% test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=1)

# Create an Random Forest classifier
model = RandomForestClassifier(n_estimators=500, random_state=42)

# Train the model
model.fit(X_train, y_train)

# Predict Output
y_pred = model.predict(X_test)

# Check
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.7704918032786885
```

Random Forest Advantages

- Because they allow us to average across many randomized Decision Trees, they are quite effective, stable, flexible, and powerful.
- Very fast at test/prediction time
- Tend to be very accurate
  - They address the biggest challenge of DTs, overfitting
  - They don’t allow any particular model’s mistakes to dominate
- Don't require much tuning

Random Forest Disadvantages

- Can be harder to interpret since they create a large number of trees and average or majority vote
  - Harder to visualize a forest
- Slower to train
  - In practice, usually not an issue – often have plenty of training time (it's test/prediction time that counts) and can parallelize the training
- Don't work well with high-dimensional sparse data
  - ex. text classification
  - They handle high dimensions well, but if sparse there are other models that perform better (ex. Naive Bayes, SVMs)
  - Tend to overfit to noise in these cases rather than the true pattern due to lack of meaningful information in each feature

We’re not looking at Naive Bayes and SVM

Here’s a little more on that than we had in Writing Assignment 5:

- https://www.ibm.com/think/topics/naive-bayes
- https://www.ibm.com/think/topics/support-vector-machine

(not on quiz)